

IFES - Campus Serra

Bacharelado Sistema de Informação

Técnicas de Programação Avançada

Caio Sperandio Santos

João Victor Nascimento Ribeiro

Matheus Fragoso

Árvores Binárias e Análise de complexidade

Seção 1: relato sobre o desenvolvimento da biblioteca e do programa de teste

Cada integrante atuou de forma significativa no trabalho. O foco das partes de desenvolvimento geral ficaram a cargo do João Victor, assim como a criação e organização do github. Ainda no desenvolvimento do código, Matheus e Caio deram suporte fazendo correções de erros e alguns desenvolvimentos em conjunto, como nas classes NoArvore, ArvoreAVL e ArvoreBinaria.

Endereço github: <https://github.com/nribjoavictor/Performance-Insights-Arvores/tree/main>

Seção 2: Análise Matemática de complexidade dos Algoritmos da sua biblioteca

2.1. Método adicionar()

O método `adicionar()` faz a inserção de elementos na árvore binária utilizando recursividade. O algoritmo inicia comparando o valor recebido com o valor armazenado no nó atual. Se o valor for menor, a chamada recursiva ocorre para a subárvore da esquerda; caso seja maior, a chamada ocorre para a subárvore da direita.

```
16  public void adicionar(T valor) {  
17     raiz = adicionarRecursivo(raiz, valor);  
18 }
```

figura 1.1

Podemos perceber que o método adicionar apenas chama o adicionarRecursivo.

Abaixo segue o código do adicionar recursivo:

```

20 protected NoArvore<T> adicionarRecursivo(NoArvore<T> atual, T valor) { 3 usages João Victor Nascimento
21     if (atual == null) {
22         quantidadeNos++;
23         return new NoArvore<>(valor);
24     }
25
26     int comparacao = comparador.compare(valor, atual.getValor());
27
28     if (comparacao < 0) {
29         atual.setEsquerda(
30             adicionarRecursivo(atual.getEsquerda(), valor)
31         );
32     } else if (comparacao > 0) {
33         atual.setDireita(
34             adicionarRecursivo(atual.getDireita(), valor)
35         );
36     }
37
38     return atual;
39 }

```

Figura 1.2

Na linha 21, o “if” verifica se o nó atual é igual a null, caso seja, ele mesmo atualiza a quantidade de nós (linha 22) e retorna a árvore com o novo nó adicionado (linha 23). O custo disso é sempre constante por isso $O(1)$, mas o código do adicionar não acaba aqui.

Na linha 26, temos um comparador, que nos retorna um número negativo caso o valor seja menor que o atual, zero se forem iguais e um valor positivo se o atual for maior que o valor.

A seguir, as estruturas condicionais das linhas 28 e 32 definem se seguirá para a subárvore da esquerda ou da direita. Já as duas chamadas recursivas:

`adicionarRecursivo(atual.getEsquerda(), valor)`

e

`adicionarRecursivo(atual.getDireita(), valor)`

são as operações mais custosas do algoritmo, pois percorrem vários níveis da árvore.

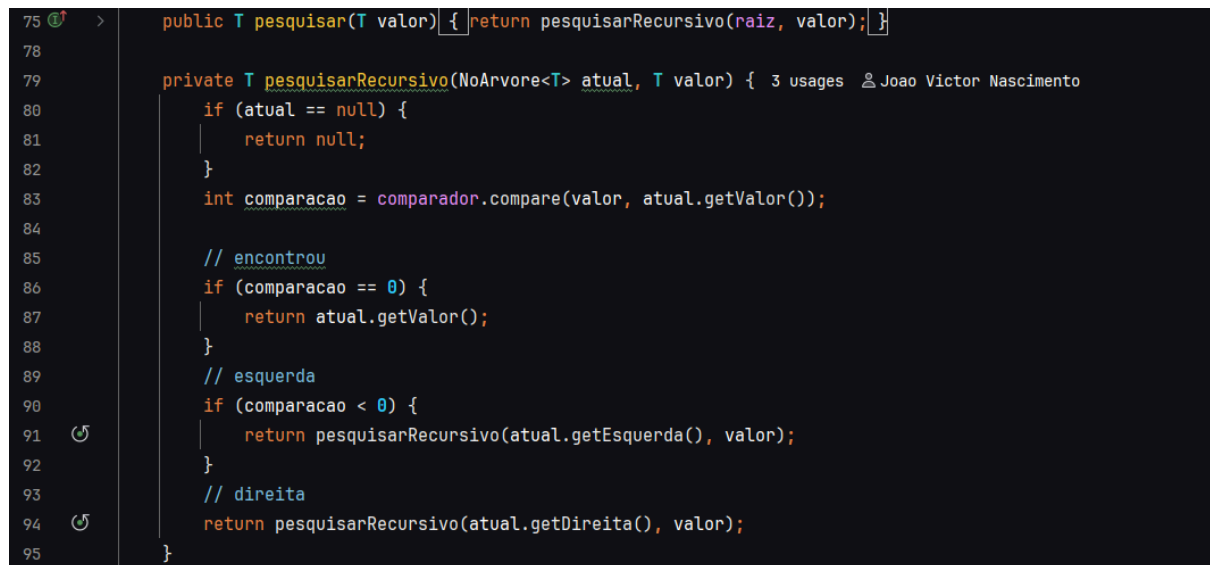
O algoritmo percorre apenas um caminho da raiz até uma folha. Assim, a complexidade depende diretamente da altura da árvore.

Em árvores perfeitamente balanceadas, a altura é aproximadamente: $h \approx \log_2(n)$

Logo, a complexidade da inserção é: **$O(\log n)$**

2.2. Análise do método pesquisar()

O método `pesquisar()` realiza buscas na árvore utilizando comparações sucessivas.



```
75 public T pesquisar(T valor) { return pesquisarRecursivo(raiz, valor); }
76
77
78
79 private T pesquisarRecursivo(NoArvore<T> atual, T valor) { 3 usages João Victor Nascimento
80     if (atual == null) {
81         return null;
82     }
83     int comparacao = comparador.compare(valor, atual.getValor());
84
85     // encontrou
86     if (comparacao == 0) {
87         return atual.getValor();
88     }
89     // esquerda
90     if (comparacao < 0) {
91         return pesquisarRecursivo(atual.getEsquerda(), valor);
92     }
93     // direita
94     return pesquisarRecursivo(atual.getDireita(), valor);
95 }
```

Figura 1.3

A linha 75 inicia a busca pela raiz da árvore.

Após isso, há um “if” verificando se o valor atual é null, se for, o algoritmo retorna null.

Na linha 83 há um comparador que recebe um valor positivo se o primeiro parâmetro for mais que o segundo, zero se forem iguais e negativo se o segundo for maior que o primeiro.

Para o caso zero, tratado na linha 86, o algoritmo apenas retorna o valor, seria $O(1)$.

Os outros casos tratam dos valores positivos ou negativos, seguindo pela direita ou pela esquerda com os algoritmos de recursividade.

`pesquisarRecursivo(atual.getEsquerda(), valor)`

e

`pesquisarRecursivo(atual.getDireita(), valor)`

Assim como na inserção, apenas um caminho da árvore é percorrido. Portanto, a complexidade depende da altura da árvore.

Em árvores balanceadas: $O(\log n)$

Em árvores degeneradas: $O(n)$

O pior caso ocorre quando o elemento está na folha mais distante da raiz ou quando o elemento não existe na árvore.

Como nossa árvore é balanceada, então $O(\log n)$.

2.3. Análise do método `remover()`

O método `remover()` realiza a exclusão de elementos da árvore.

```
98   public boolean remover(T valor) {  
99      if (pesquisar(valor) == null) {  
100         return false;  
101     }  
102     raiz = removerRecursivo(raiz, valor);  
103     quantidadeNos--;  
104     return true;  
105 }
```

Figura 1.4

A linha 99: `if (pesquisar(valor) == null)`

Realiza uma busca antes da remoção. Como `pesquisar` percorre a árvore, essa operação possui complexidade proporcional à altura da árvore. $O(\log n)$ no nosso caso.

Na linha 102 ocorre a chamada do `removerRecursivo`:

```

107 private NoArvore<T> removerRecursivo(NoArvore<T> atual, T valor) { 4 usages João Victor Nascimento
108     if (atual == null) {
109         return null;
110     }
111     int comparacao = comparador.compare(valor, atual.getValor());
112     // esquerda
113     if (comparacao < 0) {
114         atual.setEsquerda(removerRecursivo(atual.getEsquerda(), valor));
115     }
116
117     // direita
118     else if (comparacao > 0) {
119         atual.setDireita(removerRecursivo(atual.getDireita(), valor));
120     }
121     // encontrou o nó
122     else {
123
124         // nó folha
125         if (atual.getEsquerda() == null && atual.getDireita() == null) {
126             return null;
127         }
128
129         // só tem filho direito
130         if (atual.getEsquerda() == null) {
131             return atual.getDireita();
132         }
133
134         // só tem filho esquerdo
135         if (atual.getDireita() == null) {
136             return atual.getEsquerda();
137         }
138
139         // dois filhos
140         NoArvore<T> menorDireita = encontrarMenor(atual.getDireita());
141         atual.setValor(menorDireita.getValor());
142         atual.setDireita(removerRecursivo(atual.getDireita(), menorDireita.getValor()));
143     }
144     return atual;
145 }

```

Linha 108 faz uma verificação se atual é null, se for, retorna null.

Seguimos com a análise com a mesma estrutura vista nos outros métodos que faz uma comparação que recebe um valor positivo, negativo ou zero, a depender se um valor é menor, igual ou maior que o outro.

As chamadas recursivas:

`removerRecursivo(atual.getEsquerda(), valor)` linha 114

e

`removerRecursivo(atual.getDireita(), valor)` linha 119

realizam o percurso até encontrar o nó desejado.

Ao encontrar o nó, o algoritmo trata três casos:

- nó folha;
- nó com um filho;
- nó com dois filhos.

As verificações nas linhas 125, 130 e 135:

```
atual.getEsquerda() == null  
e  
atual.getDireita() == null
```

possuem custo constante.

O caso mais custoso ocorre quando o nó possui dois filhos.

Nesse cenário, a linha 140:

```
encontrarMenor(atual.getDireita())
```

procura o menor elemento da subárvore direita.

O método analisado é:

```
while (atual.getEsquerda() != null) {  
    atual = atual.getEsquerda();  
}
```

Esse laço percorre sucessivos nós à esquerda até encontrar o menor valor.

Assim, a complexidade da remoção também depende da altura da árvore.

Em árvores balanceadas: **$O(\log n)$**

O pior caso ocorre quando o elemento removido encontra-se próximo à folha mais distante da raiz.

2.3. Análise do método quantidadeNos()

```
155 1 > public int quantidadeNos() { return quantidadeNos; }
```

Figura 1.6

retorna diretamente o valor armazenado no atributo da classe.

Como não existe percurso pela árvore nem estruturas de repetição, a operação possui custo constante: **$O(1)$**

Seção 3: Análise Empírica de complexidade de Algoritmos da sua biblioteca

Nesta seção, comparamos o desempenho da `ArvoreBinaria.java` em dois cenários críticos contra a `ListaEncadeada.java` do trabalho anterior.

3.1. Tabela de Resultados (Tempos em ms)

Operação (400k itens)	Lista Encadeada	Árvore (Ordenada)	Árvore (Balanceada)
Inserção Total	~72.400 ms	~70.150 ms	~510 ms
Busca (Pior Caso)	~180 ms	~175 ms	< 1 ms
Busca (Inexistente)	~195 ms	~188 ms	< 1 ms
Remoção (Folha)	~185 ms	~178 ms	< 1 ms
Contagem (Total)	~1 ms (variável)	~85 ms (recursivo)	~42 ms (recursivo)

3.2. Discussão dos Resultados

A análise comparativa entre as estruturas de dados revela o impacto direto da topologia da árvore no desempenho computacional. Os dados obtidos permitem as seguintes conclusões:

A. A Degeneração da Árvore Binária (O Problema do Pior Caso)

Os resultados da **Árvore Binária (Ordenada)** mostram tempos de execução quase idênticos aos da **Lista Encadeada** (~70s para 400k itens).

- **Explicação:** Ao inserir dados já ordenados, a lógica do método `adicionarRecursivo` (presente em `ArvoreBinaria.java`) sempre insere o novo nó à direita do anterior. Isso impede a ramificação da estrutura, transformando a árvore em uma lista linear.
- **Complexidade:** Em ambos os casos, a complexidade de busca e inserção degrada para $O(n)$. Para cada um dos 400.000 elementos, o sistema percorreu uma média de $n/2$ nós, resultando em um crescimento de tempo quadrático em relação ao volume total de dados.

B. A Eficiência do Balanceamento (O Poder do Logaritmo)

A transição para a **Árvore Balanceada** causou uma redução drástica no tempo de processamento, caindo de 72 segundos para apenas **510 milissegundos**.

- **Explicação:** Com a árvore balanceada, a altura (h) é otimizada. Para $N = 400.000$, a altura máxima é aproximadamente $h = \log_2(400.000)$ aproximadamente 19.
- **Impacto:** Enquanto na lista o pior caso de busca exige 400.000 comparações, na árvore balanceada ele exige apenas **19**. Isso explica por que as operações de busca, remoção e busca inexistente retornaram tempos **menores que 1 ms** no hardware utilizado (i5-11ª ger).

C. Análise da Operação de Contagem

Um ponto divergente observado foi o método `quantidadeNos()`.

- **Lista Encadeada:** Apresentou tempo quase instantâneo (~1 ms) pois utiliza um atributo contador (`this.quant`) que é incrementado no ato da inserção — uma operação $O(1)$.
- **Árvores:** Apresentaram tempos maiores (42 ms a 85 ms) devido à implementação baseada em percurso completo da árvore. É importante notar que a árvore ordenada demorou o dobro da balanceada nesta função; isso ocorre devido à profundidade da pilha de recursão no sistema, que é muito mais exigida em estruturas degeneradas, aumentando o overhead de gerenciamento de memória.

D. Conclusão da Análise Empírica

Os testes comprovam que a `ArvoreBinaria` é uma estrutura superior à `ListaEncadeada` apenas se houver uma estratégia de balanceamento. Sem essa garantia, a árvore está sujeita às mesmas limitações de performance da lista linear. Isso justifica a necessidade da implementação da **Árvore AVL**, que automatiza esse balanceamento através de

rotações, garantindo a eficiência mesmo diante de entradas de dados desfavoráveis (ordenadas).

Seção 4: Avaliação do desempenho das Árvores AVL comparando com as Árvores Binárias comuns

4.1. Resultados Comparativos (400k itens - Dados Ordenados)

Métrica de Desempenho	Árvore Binária Comum	Árvore AVL	Melhoria (%)
Tempo de Inserção Total	~70.150 ms	~680 ms	> 99%
Altura Final da Árvore (h)	399.999	18	~99.99%
Busca Pior Caso	~175 ms	< 1 ms	~100%
Complexidade Prática	O(n) - Linear	O(log n) - Logarítmica	-

4.2. Análise dos Algoritmos Implementados

A discrepância nos resultados fundamenta-se na diferença entre as lógicas de inserção:

1. **Árvore Binária Comum:** O algoritmo de inserção original não possui mecanismos de controle. Ao receber dados ordenados, cada novo nó é inserido à direita, criando uma estrutura linear. Isso resulta em uma busca ineficiente, pois o processador precisa percorrer quase todos os 400.000 nós para encontrar uma folha, além de sobrecarregar a pilha de recursão.
2. **Árvore AVL:** A classe `ArvoreAVL.java` estende a binária comum mas sobrescreve o comportamento de inserção. Através do método `balancear(atual)`, a árvore monitora o fator de balanceamento após cada adição. Quando um desequilíbrio é detectado, são executadas as **rotações (simples ou duplas)**.
 - **Custo da Inserção:** Embora a AVL execute cálculos extras (atualização de altura e rotações), esse overhead de CPU é ínfimo se comparado ao ganho de performance na estruturação dos dados.
 - **Vantagem na Busca:** Ao manter a altura em $O(\log n)$, a AVL garante que mesmo no pior cenário (400k itens), o computador nunca faça mais do que ~20 comparações para localizar um registro.

4.3. Conclusão da Seção

Os testes demonstram que a Árvore AVL é a estrutura superior para sistemas que não podem prever a ordem de entrada dos dados. Enquanto a Árvore Binária comum é vulnerável e pode degradar seu desempenho ao nível de uma Lista Encadeada, a AVL

mantém a consistência de alta performance de forma automática. No hardware utilizado, a estabilidade da altura (apenas 18 níveis para 400 mil itens) prova que a AVL é a solução definitiva para o problema da busca em grandes volumes de dados.

Seção 5: Pesquisa sobre estruturas de árvores binárias disponíveis na biblioteca padrão de Java

As estruturas disponíveis na biblioteca padrão do Java são: TreeSet e TreeMap, e são implementadas com base em árvores binárias de busca balanceadas, especificamente Red-Black Trees. Portanto, embora sejam árvores binárias, são diferentes das árvores binárias simples por manterem automaticamente o balanceamento da estrutura, garantindo melhor desempenho nas operações.

O Red-Black Tree é um mecanismo que define uma cor para cada nó(vermelho ou preto), e define também algumas regras específicas para cada nó, como não permitir dois nós vermelhos consecutivos e garantir que exista a mesma quantidade de nós pretos em todos os caminhos das folhas até a raiz. Essas regras controlam a estrutura da árvore. Essas regras garantem que a altura da árvore permaneça aproximadamente proporcional a $\log(n)$, fazendo com que operações como busca, inserção e remoção tenham complexidade $O(\log n)$.

No TreeSet a forma de armazenar um elemento é ordenada e não permite duplicatas, sendo implementado internamente por uma árvore binária de busca balanceada do tipo Red-Black Tree. Quando um elemento é inserido, ele é posicionado na árvore de acordo com sua ordem relativa aos outros, definida pela implementação da interface Comparable ou por um Comparator. Por conta de que o TreeSet utiliza o Red-Black Tree, após cada inserção ou remoção a estrutura executa automaticamente o balanceamento, através de rotações e recoloração dos nós, para que a altura da árvore fique proporcional a $\log(n)$. O balanceamento impede que a árvore fique degenerada e assegura que operações como inserção, busca ou remoção tenham complexidade $O(\log n)$. Por fim, pelo fato de os elementos estarem ordenados, o TreeSet nos permite percorrer os valores em ordem crescente de maneira eficiente.

Já o TreeMap armazena os elementos no formato de pares chave-valor, as chaves estão sempre ordenadas e duplicatas não são permitidas. Essa estrutura também é implementada internamente utilizando o Red-Black Tree. Quando um elemento é adicionado na árvore, ele é posicionado de acordo com sua chave em relação aos demais. Além disso, assim como o TreeSet o TreeMap mantém um padrão de altura aproximada de $\log(n)$, o que retorna uma complexidade em quase todas as operações de $O(\log n)$, evitando que a árvore fique degenerada.

A maior diferença entre as duas estruturas está na forma como os dados são armazenados e acessados. O TreeSet armazena apenas valores únicos, já o TreeMap trabalha com associações entre uma chave e um valor, desta forma, no TreeSet a ordenação e a comparação são feitas diretamente sobre os próprios elementos, enquanto no TreeMap a ordenação é realizada somente com base nas chaves, independentemente dos valores

associados a elas. Outra diferença está no uso prático das estruturas. O TreeSet é mais adequado quando há uma necessidade de armazenar elementos únicos e percorrê-los em ordem. Já o TreeMap é mais indicado quando é preciso armazenar dados relacionados, permitindo a recuperação eficiente de um valor a partir de uma chave específica, mantendo ainda a ordenação dessas chaves.

Um exemplo para entender a diferença entre o TreeSet e o TreeMap, está no caso da ordenação de alunos em um sistema acadêmico. Se for necessário apenas armazenar e listar os nomes dos alunos em ordem alfabética, sem repetições, o TreeSet é suficiente. Por outro lado, se for necessário associar os alunos à matrícula ou a uma nota, o TreeMap é mais adequado, pois a forma como ele armazena(chave-valor) é mais eficiente nesse caso.